

ISABELA STATE UNIVERSITY

COLLEGE OF ENGINEERING, ARCHITECTURE, AND TECHNOLOGY

INFORMATION & COMMUNICATION TECHNOLOGY DEPARTMENT

IT AppDev 3

MOBILE APPLICATIONS

MODULE 2

ANDROID PLATFORM AND MODERN MOBILE DEVICE CAPABILITIES

Android Platform • Device Features • Connectivity • Sensors • Location
Security • Privacy • Permissions • Application Requirements

Course	IT AppDev 3 – Mobile Applications
Module	2
Coverage	Weeks 4–6
Learning Outcome	CLO1
Recommended Duration	3 Weeks

READY-TO-USE INSTRUCTIONAL MODULE

TABLE OF CONTENTS

This table of contents is generated as a Word field. If page numbers do not appear immediately, right-click the table and choose "Update Field."

MODULE 2 — ANDROID PLATFORM AND MODERN MOBILE DEVICE CAPABILITIES

Module Overview

Modern mobile applications are no longer limited to displaying information or accepting user input. A mobile application can communicate with other devices, determine a user's location, detect movement, capture images and audio, access biometric hardware, connect to online services, and respond intelligently to changes in the user's context.

For an application developer, however, the availability of a device feature does not automatically mean that the feature should be used. A professional mobile developer must understand the Android platform, device capabilities, connectivity, sensors, location services, security, privacy, permissions, and the relationship between technology choices and application requirements.

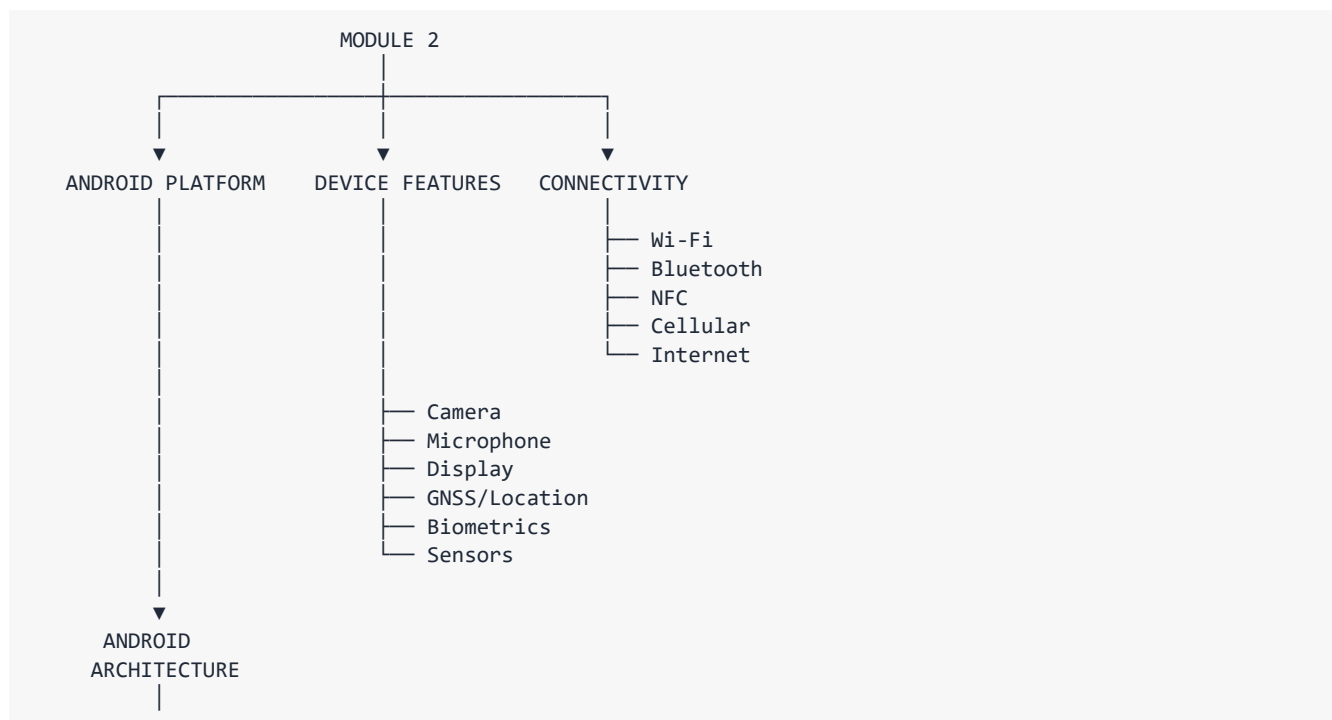
CORE IDEA

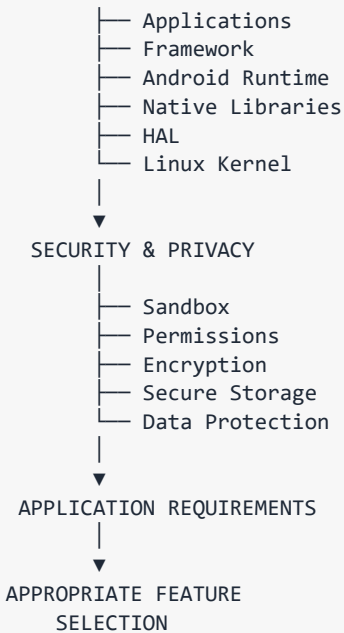
Requirement first, technology second. Select a device capability because it solves a defined application problem—not simply because the feature is available.

Module Learning Outcomes

1. Explain the history, evolution, architecture, and ecosystem of the Android platform.
2. Describe the basic and advanced hardware and software capabilities of modern mobile devices.
3. Explain how connectivity technologies, sensors, and location services support mobile application functionality.
4. Analyze the security, privacy, data-protection, and permission requirements of Android applications.
5. Evaluate appropriate mobile-device features and services for specific application requirements.

Competency Map





PART I — ANDROID PLATFORM

1.1 What is Android?

Android is an operating-system and software platform designed to support a broad range of devices and form factors. It is more accurately understood as a software stack and ecosystem than simply a smartphone operating system.

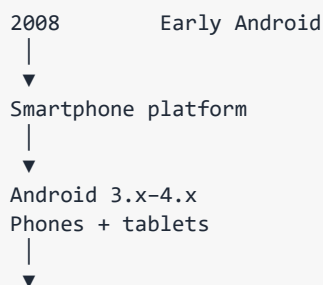
The Android platform is built as a layered stack involving the Linux kernel, hardware abstraction layer, native libraries and daemons, Android Runtime (ART), framework APIs and system services, and applications.

DEVELOPER PERSPECTIVE

Applications normally use standardized Android APIs and system services rather than communicating directly with physical hardware.

1.2 History and Evolution of Android

Android originated as a mobile software project and became part of Google's technology portfolio after Google's acquisition of Android Inc. in 2005. The first commercially released Android platform appeared in 2008. Android subsequently evolved from a smartphone-centered platform into a multi-form-factor ecosystem.





1.3 Major Android Generations

Android Generation	Approx. Era	Major Significance
Android 1.0	2008	Foundation of the commercial Android platform
Android 1.5 Cupcake	2009	Major early feature expansion
Android 2.x	2009–2011	Rapid smartphone feature and performance development
Android 3.x Honeycomb	2011	Tablet-focused platform development
Android 4.0 Ice Cream Sandwich	2011	Unified phone/tablet UI direction
Android 5.0 Lollipop	2014	ART became primary runtime; Material Design
Android 6.0 Marshmallow	2015	Runtime permissions and power improvements
Android 7.x Nougat	2016	Multi-window and productivity improvements
Android 8.x Oreo	2017	Background execution and notification controls
Android 9 Pie	2018	Adaptive intelligence and platform behavior
Android 10	2019	Privacy, gesture navigation, foldables, 5G-related support

Android Generation	Approx. Era	Major Significance
Android 11	2020	Privacy, conversations, device controls
Android 12	2021	Material You and personalization
Android 13	2022	Privacy, notifications, large-screen improvements
Android 14	2023	Security, privacy, performance, adaptive experiences
Android 15	2024	Continued large-screen, security, and performance evolution
Android 16	2025	Further adaptive, security, and platform modernization
Android 17	2026	Continued adaptive computing, privacy, security, and connectivity

1.4 Android API Levels

An API level is an integer identifying the framework API revision provided by a particular Android platform version. API levels matter because applications may operate across different Android versions.

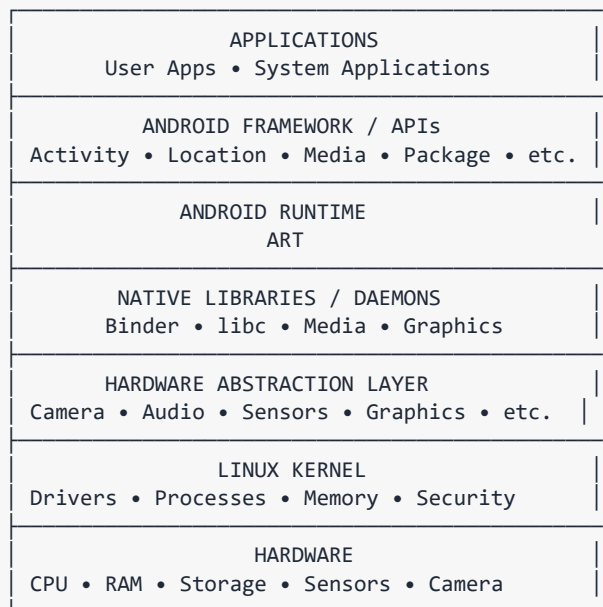
Android Version	API Level
Android 10	29
Android 11	30
Android 12	31 / 32
Android 13	33
Android 14	34
Android 15	35
Android 16	36
Android 17	37

SDK Configuration Concepts

Setting	Purpose
minSdk	Minimum Android API level the application is designed to support.
targetSdk	API level against whose platform behavior the application is designed and tested.
compileSdk	Android SDK platform used to compile the application.

PART II — ANDROID ARCHITECTURE

2.1 The Android Software Stack



The Android architecture separates application-facing APIs from hardware-specific implementation. This abstraction improves portability, security, maintainability, and compatibility across devices.

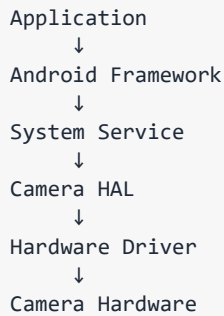
2.2 Linux Kernel

The Linux kernel is the foundation of the Android platform. It supports process management, memory management, device drivers, networking, security mechanisms, power management, and low-level hardware interaction.

Modern Android kernels are based on upstream Linux Long Term Support kernels and incorporate Android-specific changes through Android Common Kernels. The Generic Kernel Image approach separates generic kernel code from vendor-specific modules.

2.3 Hardware Abstraction Layer (HAL)

The Hardware Abstraction Layer provides standardized interfaces between higher-level Android software and hardware-specific implementations. Examples include camera, audio, sensors, Bluetooth, and graphics.



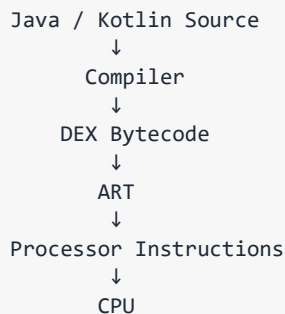
Modern Android uses AIDL for many HAL interfaces; HIDL has been deprecated for new HAL development since Android 13.

2.4 Native Libraries and Daemons

Android contains native components implemented largely in C and C++. These support graphics, media, security, Binder IPC, system utilities, and other low-level functions. Native components can provide high performance and direct system-level capabilities.

2.5 Android Runtime (ART)

The Android Runtime provides the runtime environment for Android applications. Android applications are compiled into DEX bytecode, which ART executes and optimizes using mechanisms such as ahead-of-time compilation, just-in-time compilation, garbage collection, profiling, and diagnostics.



2.6 Android Framework and System Services

The Android framework provides APIs and system services for activity management, windows, packages, resources, notifications, location, connectivity, media, sensors, permissions, accessibility, storage, and application lifecycle management.

PART III — ANDROID ECOSYSTEM

3.1 Android Open Source Project (AOSP)

The Android Open Source Project provides the Android platform source code and technical documentation. It is particularly important to device manufacturers, platform engineers, system integrators, researchers, and developers working close to the operating system.

3.2 Google Mobile Services (GMS)

AOSP should not be confused with the commercial Google service package found on compatible Google-certified Android devices. Google Mobile Services refers to Google's collection of applications and services, which can include Google Play Store, Google Play services, Google Maps, YouTube, and account integration.

REMEMBER

AOSP ≠ GMS. AOSP is the open-source Android platform; Google services are separately distributed ecosystem components.

3.3 Google Play

Google Play is a major application-distribution component of the Android ecosystem. It supports application discovery, distribution, updates, digital content, and developer publishing. Developers must follow current platform and target-API requirements when publishing applications.

3.4 Android Development Ecosystem

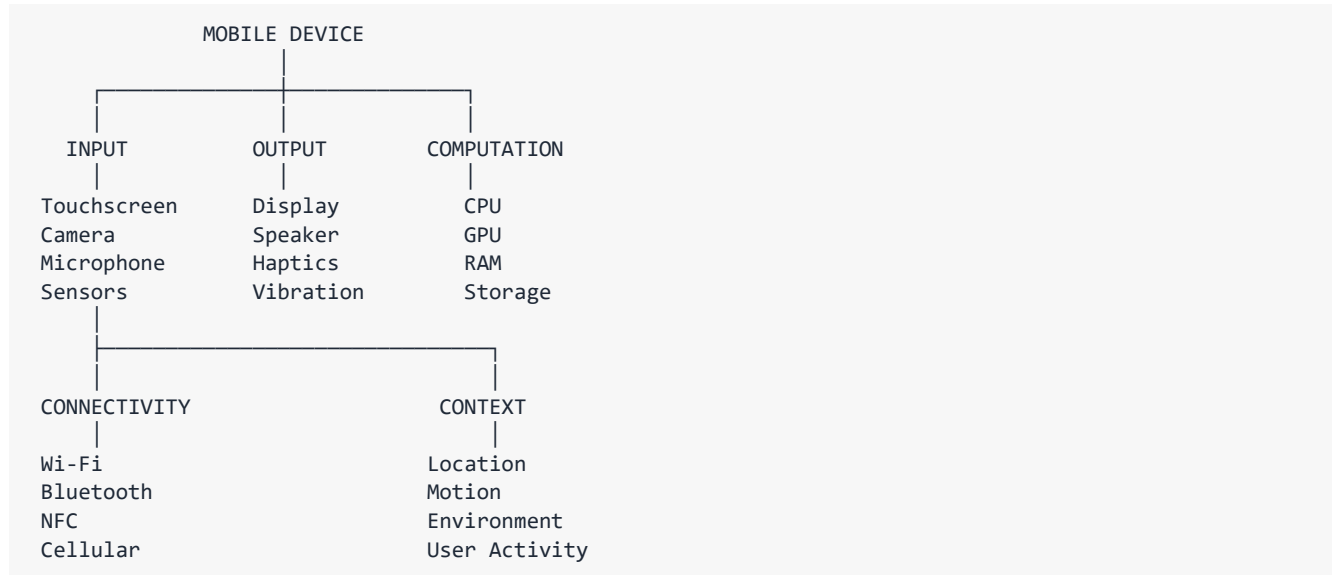
Technology / Component	Role
Kotlin	Modern statically typed language strongly promoted for Android development.
Android Studio	Official integrated development environment for Android application development.
Android SDK	APIs and tools used to build, package, debug, and test Android applications.
Gradle	Build automation and dependency-management system used by Android projects.
Jetpack	Collection of Android libraries supporting maintainable, best-practice application development.
Jetpack Compose	Modern declarative UI toolkit for Android.
ADB	Android Debug Bridge for communicating with devices and emulators.
Google Play	Major distribution and publishing platform for Android applications.

3.5 Multi-Form-Factor Android

Android now supports a broad ecosystem including smartphones, tablets, foldables, watches, televisions, automotive systems, large-screen devices, and other specialized experiences. Developers must therefore design for configuration and screen variability.

PART IV — BASIC AND ADVANCED MOBILE DEVICE FEATURES

4.1 Mobile Device Feature Categories



4.2 Basic Features

- Touchscreen — direct user interaction through tapping, swiping, scrolling, dragging, and multi-touch gestures.
- Display — screen size, resolution, density, aspect ratio, refresh rate, and orientation affect application layout.
- Camera — photography, video, QR scanning, document scanning, and computer-vision functions.
- Microphone — voice recording, communication, audio capture, and speech-related features.
- Speaker and audio — multimedia, notifications, voice communication, and accessibility.
- Storage — application data, media, files, caches, and databases.

4.3 Advanced Features

- Fingerprint and other biometric authentication
- Multi-camera and high-resolution imaging
- Depth sensing and computer-vision capabilities
- Graphics processing and AI/ML acceleration
- Near-field communication (NFC)
- Ultra-wideband on supported devices
- Advanced GNSS/location capabilities
- Health and fitness sensors
- Environmental sensors
- Foldable displays and stylus input
- High-refresh-rate displays

- Satellite connectivity on supported devices

4.4 Hardware Features vs. Software Features

Category	Examples
Hardware	Camera, GNSS, accelerometer, gyroscope, magnetometer, microphone, NFC hardware
Software / Platform	Notifications, widgets, background scheduling, accessibility services, shortcuts

4.5 Required vs. Optional Features

Developers should identify whether a device capability is essential or optional. If an application cannot function meaningfully without a capability, the developer may declare it as a required feature. If it is optional, the application should provide an alternative experience.

```
<uses-feature
  android:name="android.hardware.sensor.compass"
  android:required="true" />
```

PART V — CONNECTIVITY TECHNOLOGIES

5.1 Why Connectivity Matters

Connectivity allows mobile applications to communicate with Internet services, local networks, other mobile devices, wearables, sensors, smart-home systems, vehicles, servers, and cloud platforms.

5.2 Wi-Fi

Wi-Fi provides wireless network connectivity for Internet access, streaming, local communication, file transfer, and networked applications.

```
Android App
  ↓
Wi-Fi
  ↓
Router
  ↓
Internet
  ↓
Web / API Server
```

5.3 Cellular Connectivity

Cellular networks provide mobile Internet access and communication when Wi-Fi is unavailable. Cellular capability is particularly important for applications that must remain connected while users are moving between locations.

5.4 Bluetooth and BLE

Bluetooth supports short-range wireless communication with peripherals such as headphones, watches, vehicles, health devices, and sensors. Bluetooth Low Energy is useful for lower-power devices and intermittent data exchange.

5.5 NFC

Near Field Communication supports very short-range interactions such as contactless transactions, NFC tags, access systems, identification, and pairing.

5.6 USB

USB can support data transfer, peripheral communication, debugging, charging, and interaction with external hardware.

5.7 Emerging Connectivity

Modern Android continues to expand connectivity capabilities. Recent releases include enhancements involving device ranging, companion-device presence, Bluetooth behavior, local-network controls, and constrained satellite networking.

PART VI — SENSORS AND LOCATION SERVICES

6.1 Sensors

A sensor detects physical or environmental conditions and provides information that software can process. Android documentation groups sensors into motion, position, and environmental categories.

Sensor	Measures / Supports	Example Application
Accelerometer	Acceleration and motion	Games, orientation, activity features
Gyroscope	Rotational motion	Games, VR, stabilization, orientation
Magnetometer	Magnetic-field information	Compass and direction
Light sensor	Ambient light	Display adaptation
Pressure sensor	Atmospheric pressure	Altitude-related applications
Temperature / humidity (if available)	Environmental conditions	Environmental monitoring

6.2 Sensor Selection Principles

- Identify the information the application actually requires.
- Determine which sensor can provide that information.
- Check whether the target device contains the sensor.
- Determine an appropriate sampling rate.
- Consider battery consumption.
- Provide a fallback or alternative when appropriate.

6.3 Location-Aware Applications

Location services allow applications to provide context based on where a device is located. Common applications include navigation, ride-hailing, food delivery, weather, mapping, geofencing, emergency services, reminders, and fitness.

6.4 Sources of Location Information

- GNSS / GPS
- Wi-Fi
- Cellular networks
- Device sensors and positioning mechanisms

6.5 Location Accuracy

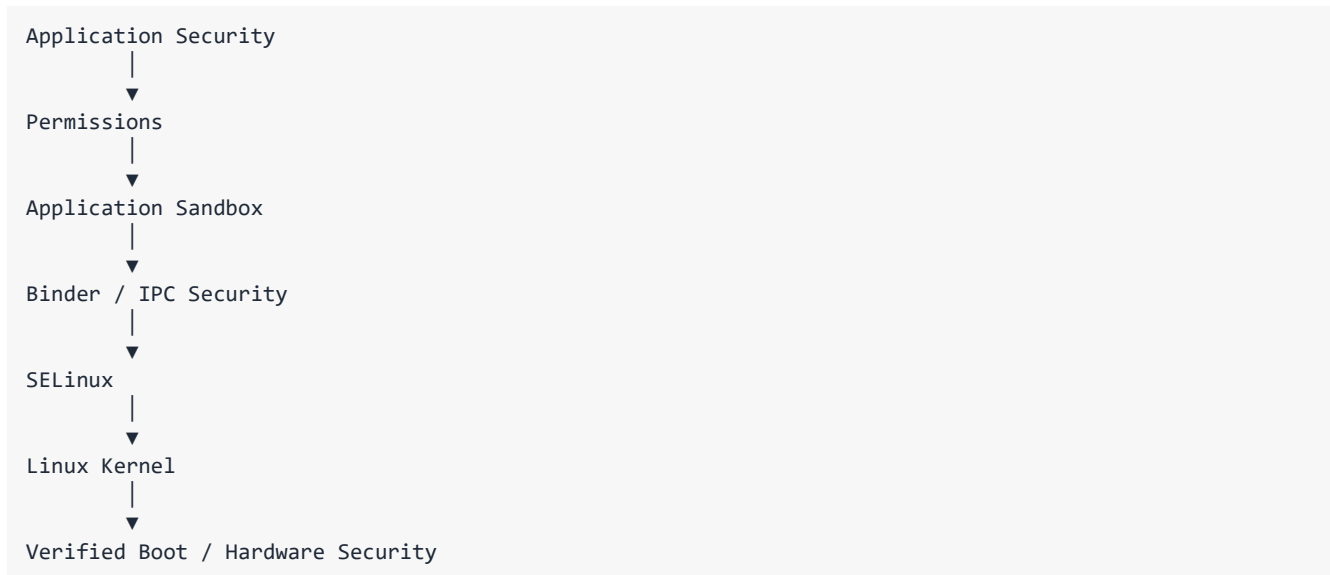
Accuracy varies with indoor/outdoor conditions, satellite visibility, network availability, device hardware, signal interference, and environmental conditions. Developers should request the level of accuracy required by the application rather than automatically requesting the highest possible precision.

6.6 Foreground and Background Location

Location can be highly sensitive. A navigation application may need location while actively being used; another application may need a one-time location only. Background location should be justified by a genuine application requirement.

PART VII — ANDROID SECURITY, PRIVACY, AND PERMISSIONS

7.1 Android Security as an Architectural Principle



Android security is implemented across multiple layers, including process isolation, application sandboxing, permissions, secure IPC, SELinux, Verified Boot, cryptographic facilities, and hardware-backed security where available.

7.2 Application Sandbox

Android applications are isolated from one another. The sandbox helps prevent an application from freely accessing another application's private files and data.

7.3 SELinux

Security-Enhanced Linux provides mandatory access control, adding another security layer beyond traditional discretionary access controls.

7.4 Verified Boot

Verified Boot helps establish the integrity of software loaded during device startup by cryptographically verifying software components through the boot chain.

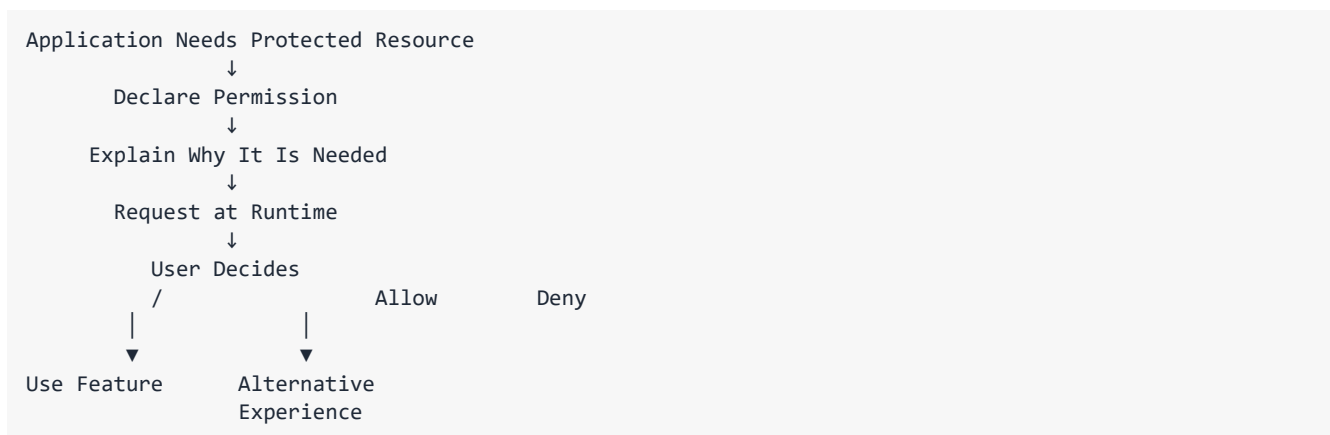
7.5 Android Permissions

Permissions control access to protected or sensitive resources. Examples include camera, microphone, location, contacts, nearby devices, and other protected information.

LEAST PRIVILEGE

Request only the permissions necessary for the application's legitimate functionality. A permission should be justified by a specific user-facing feature.

7.6 Permission Workflow



7.7 Security vs. Privacy

Concept	Core Question
Security	Can unauthorized people access, modify, or destroy the data?
Privacy	Should the application collect, use, retain, or share this data at all?

A secure application can still have poor privacy practices if it collects unnecessary information.

7.8 Data Minimization

Developers should collect only information necessary for the application's legitimate purpose. Data minimization reduces privacy risk, storage requirements, attack surface, and unnecessary processing.

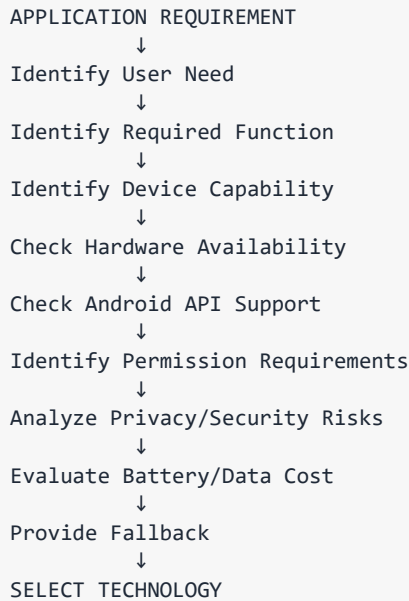
PART VIII — EVALUATING FEATURES FOR APPLICATION REQUIREMENTS

8.1 The Developer's Decision Problem

START WITH THE PROBLEM

Do not begin with “What Android feature can I use?” Begin with “What problem must the application solve?” Then determine which technology is appropriate.

8.2 Feature Selection Framework



8.3 Feature Evaluation Matrix

Application Requirement	Appropriate Feature / Service	Key Considerations
Navigate a user	Location + Maps	Permission, accuracy, battery
Scan a QR code	Camera	Permission, lighting, device availability
Connect to smartwatch	Bluetooth / BLE	Pairing, nearby-device access
Contactless transaction	NFC	Compatibility, security
Detect device movement	Accelerometer / Gyroscope	Sampling rate, battery
Determine direction	Magnetometer	Availability, calibration
Record voice	Microphone	Permission, privacy
Take photographs	Camera	Permission, storage

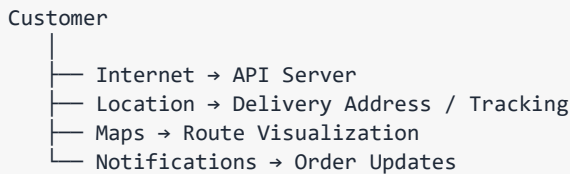
Application Requirement	Appropriate Feature / Service	Key Considerations
Track outdoor activity	Location + Sensors	Battery, background access
Connect to online server	Wi-Fi / Cellular	Network availability, secure communication
Authenticate user	Biometrics	Hardware capability and fallback
Detect environment	Environmental sensors	Sensor availability

PART IX — APPLICATION CASE STUDIES

Case Study 1 — Food Delivery Application

Requirement: A customer wants to order food and track the delivery rider.

- Internet connectivity
- Location
- Maps
- Notifications
- User authentication



Important considerations include location privacy, network reliability, battery consumption, secure communication, and permission handling.

Case Study 2 — Fitness Application

Requirement: An application records walking activity and displays the user's route.

- Location
- Accelerometer
- Activity recognition
- Storage
- Bluetooth for wearable integration

The developer must determine whether continuous or background location is genuinely required.

Case Study 3 — Smart Classroom Application

Requirement: A university application detects nearby classroom devices and exchanges information.

- Wi-Fi
- Bluetooth / BLE
- Local network communication
- QR code

- NFC

The technologies should be compared using range, power consumption, data rate, user interaction, security, permissions, and compatibility.

Case Study 4 — Emergency Application

Requirement: An emergency application sends an alert containing the user's location and optionally a photo.

- Location
- Cellular / Internet connectivity
- Notifications
- Camera
- Authentication
- Secure storage / transmission

CRITICAL QUESTION

What is the minimum information required to perform the emergency function? This is a security and privacy decision as much as a technical decision.

PART X — LEARNING ACTIVITIES

Activity 1 — Android Architecture Diagram

Objective: Demonstrate understanding of the Android software stack.

Instructions: On A4 bond paper, create a professionally labeled Android architecture diagram. Include Applications, Android Framework, Android Runtime, Native Libraries/Daemons, HAL, Linux Kernel, and Hardware. For every layer, provide a definition, two examples, and its role.

Guide Question: How does an Android application communicate with hardware without directly controlling the hardware?

Activity 2 — Mobile Device Feature Inventory

Objective: Identify and classify mobile-device capabilities.

Use an Android smartphone or device specification sheet. Identify at least 15 features and classify each as hardware, software, connectivity, sensor, location, or security.

Feature	Category	Available?	Possible Application Use
Camera	Hardware	Yes / No	Photo capture
GNSS	Location	Yes / No	Navigation
Accelerometer	Sensor	Yes / No	Motion detection
Bluetooth	Connectivity	Yes / No	Device communication
Biometric sensor	Security	Yes / No	Authentication

Activity 3 — Connectivity Comparison

Compare Wi-Fi, Bluetooth, Bluetooth Low Energy, NFC, and Cellular using range, data rate, power consumption, typical applications, security considerations, and appropriate use cases.

Activity 4 — Sensor Investigation

Identify the sensors available on an Android device. For each sensor, identify what it measures, an application that can use it, whether it is essential or optional, and possible battery implications.

Activity 5 — Permission Analysis

Analyze a camera app, weather app, fitness app, food-delivery app, flashlight app, and calculator app. Identify likely permissions, why each is needed, sensitivity, denial behavior, and alternatives.

Application	Possible Permission	Why Needed	If Denied
Camera	Camera	Capture images/video	Camera feature unavailable
Weather	Location	Current-area weather	Manual location entry
Fitness	Location / sensors	Route and activity	Reduced tracking
Food delivery	Location	Delivery address/tracking	Manual address
Flashlight	Usually none for basic function	Torch control may use camera hardware	Feature may be unavailable on some devices
Calculator	Usually none	Core calculation	No protected-resource dependency

Activity 6 — Scenario-Based Feature Selection

Scenario: A university wants a Campus Safety Mobile Application that allows students to report emergencies, send location, optionally capture a photo, notify authorized personnel, work over available networks, and protect personal information.

Task: Evaluate Location, Camera, Internet, Cellular connectivity, Notifications, Authentication, Storage, Security, and Permissions. Explain which are required, optional, or conditional.

PART XI — GROUP ACTIVITY: CHOOSE THE RIGHT MOBILE TECHNOLOGY

Divide the class into groups. Each group receives one application requirement.

Group	Application Requirement
A	Ride-hailing application
B	Fitness tracker
C	Smart-home controller

Group	Application Requirement
D	Campus navigation system
E	Emergency reporting application
F	Digital attendance system

Each group must identify: required device features, sensors, connectivity, location services, permissions, privacy concerns, security concerns, device compatibility concerns, and fallback mechanisms.

PART XII — PERFORMANCE TASK

Android Mobile Feature and Service Analysis

Select one proposed mobile application such as campus navigation, food delivery, fitness tracking, emergency reporting, smart classroom, agricultural monitoring, health monitoring, transportation, tourism, or e-commerce.

Required Analysis

12. Application Purpose — Explain the problem the application solves.
13. Target Users — Identify intended users.
14. Required Device Features — Identify needed hardware/software capabilities.
15. Connectivity — Identify appropriate connectivity technologies.
16. Sensors — Identify required sensors.
17. Location — Determine whether location is necessary.
18. Permissions — Identify permissions that may be required.
19. Security — Identify risks and controls.
20. Privacy — Identify sensitive information and data-minimization strategies.
21. Compatibility — Identify devices/features that may limit availability.
22. Fallback — Explain what happens when a feature is unavailable.
23. Final Recommendation — Justify why the selected features are appropriate.

Performance Task Rubric

Criteria	4 – Excellent	3 – Proficient	2 – Developing	1 – Beginning
Requirement Analysis	Complete and precise	Mostly complete	Partially complete	Limited
Device Feature Selection	Highly appropriate	Appropriate	Some mismatches	Inappropriate
Connectivity Analysis	Thorough	Adequate	Limited	Minimal
Sensor / Location Analysis	Accurate	Mostly accurate	Some errors	Inaccurate
Security Analysis	Comprehensive	Appropriate	Limited	Insufficient

Criteria	4 – Excellent	3 – Proficient	2 – Developing	1 – Beginning
Privacy Analysis	Strong	Adequate	Limited	Missing
Permission Analysis	Correct and justified	Mostly correct	Some errors	Incorrect
Compatibility	Thorough	Adequate	Limited	Missing
Technical Reasoning	Excellent	Good	Developing	Weak
Presentation	Professional	Clear	Acceptable	Poor

Suggested performance standard: 30/40 or 75% — Proficient.

PART XIII — REVIEW QUESTIONS

Android Platform

24. Why is Android considered a software platform?
25. What is AOSP?
26. What is an API level?
27. What is the purpose of the Android Runtime?
28. What is the purpose of a HAL?
29. Why is the Linux kernel important to Android?
30. What services does the Android framework provide?
31. Why must Android applications account for different device configurations?

Mobile Device Features

32. What are basic mobile-device features?
33. What are advanced mobile-device features?
34. What is the difference between a hardware and software feature?
35. Why should developers distinguish required and optional features?

Connectivity

36. What is Wi-Fi?
37. What is Bluetooth?
38. What is BLE?
39. What is NFC?
40. When would cellular connectivity be necessary?
41. How should a developer select a connectivity technology?

Sensors and Location

42. What is an accelerometer?
43. What is a gyroscope?
44. What is a magnetometer?
45. What are environmental sensors?
46. What is GNSS/GPS?
47. What is geofencing?
48. Why is location considered sensitive information?

Security and Privacy

49. What is application sandboxing?
50. What is SELinux?
51. What is Verified Boot?
52. Why are Android permissions necessary?
53. What is data minimization?
54. What is the difference between security and privacy?
55. Why should an application provide alternatives when a user denies permission?

Application Requirements

56. Why should feature selection begin with application requirements?
57. How do device capabilities affect application design?
58. Why should developers consider battery consumption?
59. Why should developers provide fallback mechanisms?
60. Why is compatibility testing important?
61. How should a developer determine whether a feature is essential?

Short Quiz

Part A — Identification

62. The open-source project containing Android platform source code.
63. The runtime environment used to execute Android application bytecode.
64. The layer that provides standardized interfaces to device hardware.
65. The foundation of the Android operating system.
66. The official IDE for Android development.
67. A modern programming language strongly promoted for Android development.
68. A collection of Android libraries supporting best practices.
69. The modern declarative UI toolkit for Android.
70. The integer identifier associated with an Android platform API revision.
71. Google's major application distribution platform for Android.

Part B — Multiple Choice

11. Which component is at the foundation of the Android platform?

- A. Jetpack
- B. Linux Kernel
- C. Android Studio
- D. Google Play

12. What is the primary purpose of the HAL?

- A. Build application interfaces
- B. Manage Google accounts
- C. Abstract hardware-specific implementations
- D. Compile Kotlin code

13. Which runtime replaced Dalvik as the primary runtime beginning with Android 5.0?

- A. JVM
- B. ART

- C. CLR
- D. Node.js

14. Which technology is used to build modern declarative Android UIs?

- A. Jetpack Compose
- B. ADB
- C. Gradle
- D. HAL

15. Which organization maintains the Android Open Source Project?

- A. Microsoft
- B. Apple
- C. Google
- D. Oracle

Applied Assessment Scenario

A developer is creating a weather application. The application needs to display current weather, weather based on the user's current location, weather alerts, and forecast information.

Questions

72. Which connectivity technologies are required?
73. Is location required?
74. Is precise location necessarily required?
75. Should background location be used?
76. Which permissions may be necessary?
77. What privacy concerns exist?
78. What should happen if the user denies location permission?
79. What should happen if the device has no Internet connection?
80. What data should be stored locally?
81. Which device feature is essential and which features are optional?

PART XIV — KEY TAKEAWAYS

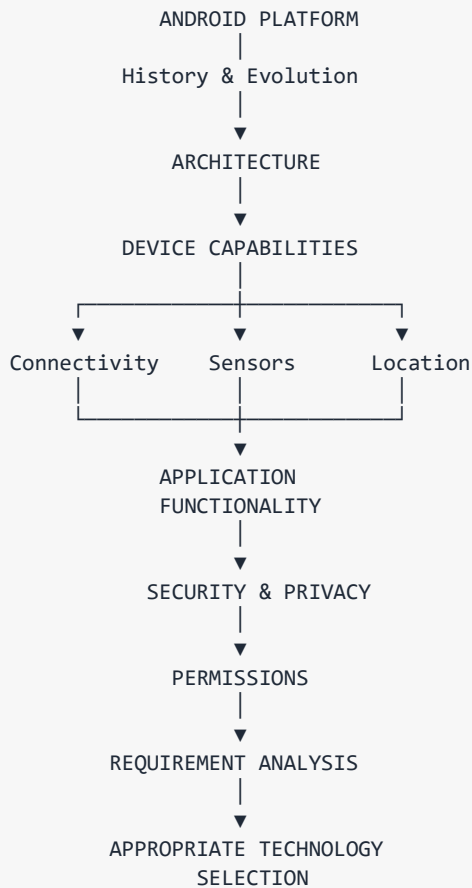
82. Android is a layered software platform.
83. The Linux kernel provides the foundation of the Android platform.
84. HALs abstract hardware-specific implementations from higher Android layers.
85. ART provides the runtime environment for Android applications.
86. Android devices do not all contain the same hardware capabilities.
87. Connectivity allows applications to communicate with networks and other devices.
88. Sensors provide information about motion, position, and the physical environment.
89. Location services enable location-aware application functionality.
90. Permissions protect sensitive device capabilities and user information.
91. Security and privacy must be considered throughout application design.
92. Developers should collect only the data they actually need.
93. The best device feature is the one that appropriately satisfies the application requirement—not necessarily the most advanced feature available.

Professional Development Checklist

- ☐ What user requirement does this feature address?

- ☐ Is the feature actually necessary?
- ☐ Is the feature available on the target device?
- ☐ Which Android API provides the feature?
- ☐ What Android version is required?
- ☐ Is a permission required?
- ☐ Is the permission sensitive?
- ☐ How will the application explain the permission request?
- ☐ What happens if the user denies permission?
- ☐ What happens if the hardware is unavailable?
- ☐ Does the feature consume significant battery?
- ☐ Does it require network connectivity?
- ☐ Does it process personal or sensitive information?
- ☐ How will sensitive data be protected?
- ☐ Is background access genuinely necessary?
- ☐ Is there a fallback mechanism?
- ☐ Has the feature been tested on multiple device configurations?
- ☐ Does the feature improve the user's experience enough to justify its cost and risk?

PART XV — MODULE SYNTHESIS



MODULE PRINCIPLE

REQUIREMENT FIRST, TECHNOLOGY SECOND. A good Android developer determines what the user needs, then selects the simplest, safest, most appropriate capability that satisfies that need.

REFERENCES AND CURRENT TECHNICAL RESOURCES

For platform-specific details, students should prioritize current official Android and AOSP documentation because API levels, platform behavior, development tools, and publishing requirements change over time.

- **Android Developers — Platform Architecture:** <https://developer.android.com/guide/platform/index.html>
- **Android Developers — Platform Releases:** <https://developer.android.com/about/versions>
- **Android Developers — API Levels:** <https://developer.android.com/guide/topics/manifest/uses-sdk-element>
- **Android Developers — Architecture Recommendations:** <https://developer.android.com/topic/architecture/recommendations>
- **Android Developers — Kotlin for Android:** <https://developer.android.com/kotlin>
- **Android Developers — Android Studio:** <https://developer.android.com/studio>
- **Android Developers — Jetpack:** <https://developer.android.com/jetpack>
- **Android Open Source Project — AOSP:** <https://source.android.com/>
- **AOSP — Architecture:** <https://source.android.com/docs/core/architecture>
- **AOSP — Kernel:** <https://source.android.com/docs/core/architecture/kernel>
- **AOSP — HAL:** <https://source.android.com/docs/core/architecture/hal>
- **AOSP — Security:** <https://source.android.com/docs/security>

Institutional Reference

Isabela State University – IT AppDev 3 Mobile Applications Course Syllabus, 1st Semester, SY 2026–2027, Revision 05, July 13, 2026. The syllabus identifies the Android platform, architecture, ecosystem, device features, connectivity, sensors, location, security, privacy, and permissions as core course content and places these competencies under CLO1 and Weeks 4–6.

MODULE 2 — END

Android Platform and Modern Mobile Device Capabilities